model_evaluation_metrics.ipynb

Quality metrics for classification

Confusion matrix

TP = true positive

FP = false positive

TN = true negative

FN = false negative

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import RobustScaler, PolynomialFeatures
import matplotlib.pyplot as plt
import matplotlib as mpl
from tqdm import tqdm, trange

np.random.seed(42)

cmap = mpl.colors.ListedColormap(['#992622', '#005AAA'])
```

```
def gen_dataset(n_positive, n_negative):
    outer_circ_x = np.cos(np.linspace(0, np.pi, n_negative))
    outer_circ_y = np.sin(np.linspace(0, np.pi, n_negative))
    inner_circ_x = 1 - np.cos(np.linspace(0, np.pi, n_positive))
    inner_circ_y = 1 - np.sin(np.linspace(0, np.pi, n_positive)) - .5

    X = np.vstack([np.append(outer_circ_x, inner_circ_x),
                   np.append(outer_circ_y, inner_circ_y)]).T
    y = np.hstack([np.zeros(n_negative, dtype=np.intp),
                   np.ones(n_positive, dtype=np.intp)])

    X += np.random.normal(scale=0.4, size=X.shape)

    return X, y

X, y = gen_dataset(200, 200)
plt.figure(figsize=(12, 8))
plt.scatter(*X.T, c=y, s=40, cmap=cmap);
```

```
def plot_prediction(model, nx=100, ny=100, xmin=-2.5, xmax=3.5, ymin=-2.5, ymax=2.5):
    xx, yy = np.meshgrid(
      np.linspace(xmin, xmax, nx),
      np.linspace(ymin, ymax, ny)
    )

    preds = model.predict_proba(np.c_[xx.ravel(), yy.ravel()])[:,0].reshape(xx.shape)
    plt.contour(xx, yy, preds, levels=[0.5], colors='k', linewidths=3)
```

Exercise 1: plotting TP, FP, TN and FN

Modify the code below to distinguish true positives, false positives, true negatives and false negatives on the plot. Add legend.

```
model = make_pipeline(
    RobustScaler(),
    PolynomialFeatures(3, include_bias=False),
    LogisticRegression()
)

model.fit(X, y)
preds = model.predict(X)

plt.figure(figsize=(15, 10))
plt.scatter(*X.T, c=np.array([y==preds]).ravel(), s=20, cmap=cmap);
plot_prediction(model)
```

Exercise 2: accuracy, precision, recall, F1 score

$$\mathbf{accuracy} = \frac{\mathbf{TP} + \mathbf{TN}}{\mathbf{TP} + \mathbf{TN} + \mathbf{FP} + \mathbf{FN}}$$

$$\mathbf{precision} = \frac{\mathbf{TP}}{\mathbf{TP} + \mathbf{FP}}$$

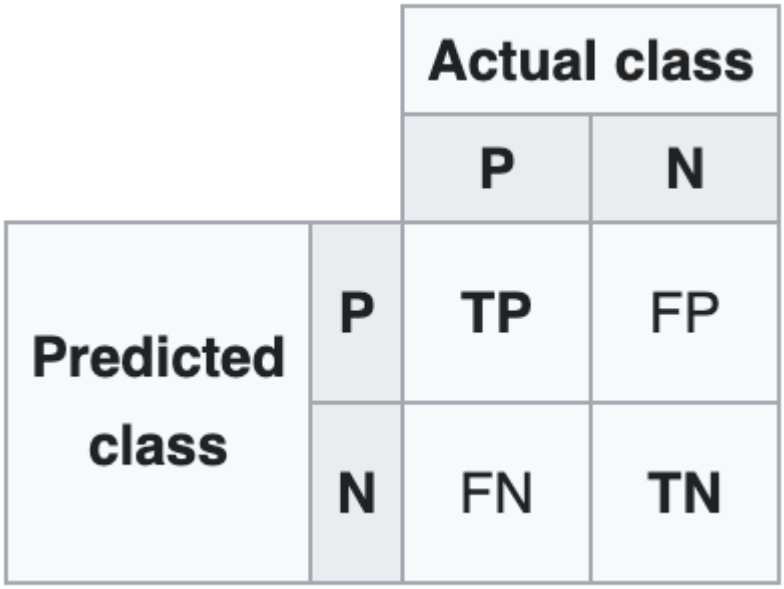
$$\mathbf{recall} = \frac{\mathbf{TP}}{\mathbf{TP} + \mathbf{FN}}$$

$$F_1 = \frac{2 \cdot \mathbf{precision} \cdot \mathbf{recall}}{\mathbf{precision} + \mathbf{recall}}$$

Implement the calculation of quality metrics in the function below.

calculate_metrics(dummy_y_true, dummy_y_pred)

(skmetrics.accuracy_score(dummy_y_true, dummy_y_pred), skmetrics.precision_score(dummy_y_true, dummy_y_pred), skmetrics.recall_score(dummy_y_true, dummy_y_pred), skmetrics.f1_score(dummy_y_true, dummy_y_pred))




```
def calculate_metrics(y_true, y_pred):
    '''
    Function to calculate quality metrics.

    arguments:
    - y_true - numpy array of true labels (0 - negative class, 1 - positive class)
    - y_pred - numpy array of model predictions (0 - negative class, 1 - positive class)

    returns a tuple containing the calculated metrics:
    `(accuracy, precision, recall, f1)`
    '''
    TP = np.sum(y_true[y_true == 1] == y_pred[y_true == 1])
    TN = np.sum(y_true[y_true == 0] == y_pred[y_true == 0])
    FN = np.sum(y_true[y_pred == 0] != y_pred[y_pred == 0])
    FP = np.sum(y_true[y_pred == 1] != y_pred[y_pred == 1])

    # YOUR CODE: use only numpy operations to calculate the metrics
    accuracy = (TP + TN) * 1. / (TP + TN + FP + FN)
    precision = TP * 1. / (TP + FP)
    recall = TP * 1. / (TP + FN)
    f1 = 2 * precision * recall / (precision + recall)

    return [accuracy, precision, recall, f1]


# Automatic checks:
import sklearn.metrics as skmetrics

dummy_y_true = np.array([0, 0, 0, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 0])
dummy_y_pred = np.array([1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0])

dummy_accuracy, dummy_precision, dummy_recall, dummy_f1 = \
  calculate_metrics(dummy_y_true, dummy_y_pred)
assert dummy_accuracy == skmetrics.accuracy_score(dummy_y_true, dummy_y_pred), 'Wrong accuracy'
assert dummy_precision == skmetrics.precision_score(dummy_y_true, dummy_y_pred), 'Wrong
precision'
assert dummy_recall == skmetrics.recall_score(dummy_y_true, dummy_y_pred), 'Wrong recall'
assert dummy_f1 == skmetrics.f1_score(dummy_y_true, dummy_y_pred), 'Wrong F1 score'
print("All good!")
```

Exercise 3: metrics as the function of class imbalance

Let's see how the metrics above behave when the classes are imbalanced.

For each α in `pos_class_fractions` below generate train and test datasets with $N = 50000$ points each, such that `n_positive` $= \alpha N$, `n_negative` $= (1 - \alpha)N$ (**don't foget to round and convert these numbers to integers!**). Train and evaluate the model on the generated data, plot test metrics (accuracy, precision, recall and F1) as a function of α on the same plot.

```
pos_class_fractions = np.linspace(0.01, 0.99, 30)
```

```
N = 50000
qual = []
qual_check = []
pos_class_fractions = np.linspace(0.01, 0.99, 30)
for alpha in pos_class_fractions:
    X_train, y_train = gen_dataset(int(N * alpha), int(N * (1 - alpha)))
    X_test, y_true = gen_dataset(int(N * alpha), int(N * (1 - alpha)))
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    qual.append(calculate_metrics(y_true, y_pred))
    qual_check.append([skmetrics.accuracy_score(y_true, y_pred),
                       skmetrics.precision_score(y_true, y_pred),
                       skmetrics.recall_score(y_true, y_pred),
                       skmetrics.f1_score(y_true, y_pred)])
```

```
X_train, y_train = gen_dataset(500, 49500)
X_test, y_true = gen_dataset(500, 49500)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
TP = np.sum(y_true[y_true == 1] == y_pred[y_true == 1])
TN = np.sum(y_true[y_true == 0] == y_pred[y_true == 0])
FN = np.sum(y_true[y_pred == 0] != y_pred[y_pred == 0])
FP = np.sum(y_true[y_pred == 1] != y_pred[y_pred == 1])
```

```
accuracy = (TP + TN) * 1. / (TP + TN + FP + FN)
precision = TP * 1. / (TP + FP)
recall = TP * 1. / (TP + FN)
f1 = 2 * precision * recall / (precision + recall)
```

```
recall, precision, f1, accuracy
```

```
X_train, y_train = gen_dataset(49500, 500)
X_test, y_true = gen_dataset(49500, 500)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
TP = np.sum(y_true[y_true == 1] == y_pred[y_true == 1])
TN = np.sum(y_true[y_true == 0] == y_pred[y_true == 0])
FN = np.sum(y_true[y_pred == 0] != y_pred[y_pred == 0])
FP = np.sum(y_true[y_pred == 1] != y_pred[y_pred == 1])
```

```
accuracy = (TP + TN) * 1. / (TP + TN + FP + FN)
precision = TP * 1. / (TP + FP)
recall = TP * 1. / (TP + FN)
f1 = 2 * precision * recall / (precision + recall)
```

```
recall, precision, f1, accuracy
```

```
qual = np.array(qual)
```

```
y_pred == y_true
```

```
49500/50000.
```

В предыдущем примере: 49 500 объектов класса 1 и 500 объектов класса 0.

```
plt.figure(figsize=(16, 10))
plt.grid()
plt.plot(pos_class_fractions, qual[:, 0], label="accuracy")
plt.plot(pos_class_fractions, qual[:, 1], label="precision")
plt.plot(pos_class_fractions, qual[:, 2], label="recall")
plt.plot(pos_class_fractions, qual[:, 3], label="f1-measure")
plt.legend(loc='best')
plt.xlabel("fractions of positive class")
plt.ylabel("metrics")
```

ROC curve

For a given model that gives the positive class score with $\hat{f}(x)$, prediction is obtained by thresholding this score with some threshold T:

$$\hat{y} = \mathbb{I}\left\{\hat{f}(x) \geq T\right\}$$

Then, true and false positive rates can be defined as functions of threshold.

True positive rate:

$$\mathrm{TPR}(T) = \frac{\mathrm{TP}(T)}{\mathrm{TP}(T) + \mathrm{FN}(T)}$$

False positive rate:

$$\mathrm{FPR}(T) = \frac{\mathrm{FP}(T)}{\mathrm{FP}(T) + \mathrm{TN}(T)}$$

ROC curve - TPR as a function of FPR.

Check out this nice demo: http://arogozhnikov.github.io/2015/10/05/roc-curve.html (http://arogozhnikov.github.io/2015/10/05/roc-curve.html)

```
X_train, y_train = gen_dataset(200, 200)
X_test, y_test = gen_dataset(200, 200)

model = make_pipeline(
    RobustScaler(),
    PolynomialFeatures(3, include_bias=False),
    LogisticRegression()
)

model.fit(X_train, y_train)
preds_test = model.predict(X_test)
proba_test = model.predict_proba(X_test)[:, 1]

from sklearn.metrics import roc_curve

fpr, tpr, _ = roc_curve(y_test, preds_test)
plt.plot(fpr, tpr, label='discrete output');
fpr, tpr, _ = roc_curve(y_test, proba_test)
plt.plot(fpr, tpr, label='probability output');
plt.legend()
plt.xlabel('false positive rate')
plt.ylabel('true positive rate');
```

```
y_true = y_test
y_pred = preds_test
```

```
TP = np.sum(y_true[y_true == 1] == y_pred[y_true == 1])
TN = np.sum(y_true[y_true == 0] == y_pred[y_true == 0])
FN = np.sum(y_true[y_pred == 0] != y_pred[y_pred == 0])
FP = np.sum(y_true[y_pred == 1] != y_pred[y_pred == 1])
```

```
fpr, tpr, _ = roc_curve(y_test, preds_test)
```

```
fpr, tpr
```

```
FP * 1. / (FP + TN)
```

```
TP * 1. / (TP + FN)
```

Another metric - area under the ROC curve ("ROC AUC", or sometimes just "AUC").

- AUC = 1.0 – perfect classification
- AUC = 0.5 – random ranking in expectation; values below 0.5 are possible
- AUC = 0.0 – completely reversed ranking (converts to perfect ranking by swapping the labels $\mathbf{1} \leftrightarrow \mathbf{0}$)

```
from sklearn.metrics import roc_auc_score

print('AUC (discrete):', roc_auc_score(y_test, preds_test))
print('AUC (probability):', roc_auc_score(y_test, proba_test))
```

AUC

```
import pandas as pd
```

```
d = pd.DataFrame({"true": [0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1],
                  "pred": [0.2, 0.7, 0.6, 0.1, 0.5, 0.3, 0.5, 0.4, 0.6, 0.1, 0.1, 0.9, 0.8,
0.1, 0.5, 0.3, 0.2]})
```

```
d
```

```
d = d.sort_values("pred", ascending=False)
```

```
d.values[:, 1].astype(int)
```

```
54.5/66
```

```
roc_auc_score(d["true"], d["pred"])
```

```
fpr, tpr, _ = roc_curve(d["true"], d["pred"])
plt.grid()
plt.plot(fpr, tpr, label='probability output');
```

```
d
```

```
def I_a(a_i, a_j):
    if a_i > a_j:
        return 0.
    elif a_i < a_j:
        return 1.
    return 0.5

def I_y(y_i, y_j):
    if y_i >= y_j:
        return 0.
    return 1.
```

```
nominator = 0
denominator = 0
for i in range(d.shape[0]):
    for j in range(d.shape[0]):
        nominator += I_y(d["true"][i], d["true"][j]) * I_a(d["pred"][i], d["pred"][j])
        denominator += I_y(d["true"][i], d["true"][j])
```

```
nominator / denominator
```

```

```